418. Sentence Screen Fitting

Medium 435 215 Add to List Share

Given a `rows x cols` screen and a sentence represented by a list of **non-empty** words, find **how many times** the given sentence can be fitted on the screen.

Note:

1. A word cannot be split into two lines.
2. The order of words in the sentence must remain unchanged.
3. Two consecutive words **in a line** must be separated by a single space.
4. Total words in the sentence won't exceed 100.
5. Length of each word is greater than 0 and won't exceed 10.
6. 1 ≤ rows, cols ≤ 20,000.

Example 1:

```
Input:
rows = 2, cols = 8, sentence = ["hello", "world"]

Output:
1

Explanation:
hello---
world---

The character '-' signifies an empty space on the screen.
```

Example 2:

```
Input:
rows = 3, cols = 6, sentence = ["a", "bcd", "e"]

Output:
2

Explanation:
a-bcd-
e-a---
bcd-e-

The character '-' signifies an empty space on the screen.
```

Example 3:

```
Input:
rows = 4, cols = 5, sentence = ["I", "had", "apple", "pie"]

Output:
1

Explanation:
I-had
apple
pie-I
had--

The character '-' signifies an empty space on the screen.
```

Accepted 42,614 Submissions 130,893

Seen this question in a real interview before? Yes No

Contributor

Companies

Related Topics

Java

Autocomplete

Need more space? Try using full screen mode!

Got it!

```java
class Solution {
    public int wordsTyping(String[] sentence, int rows, int cols) {
        int[] dp = new int[sentence.length];
        int n = sentence.length;
        for(int i = 0, prev = 0, len = 0; i < sentence.length; ++i) {
            // remove the length of previous word and space
            if(i != 0 && len > 0) len -= sentence[i - 1].length() + 1;
            // calculate the start of next line.
            // it's OK the index is beyond the length of array so that
            // we can use it to count how many words one row has.
            while(len + sentence[prev % n].length() <= cols) len += sentence[prev++ % n].length() + 1;
            dp[i] = prev;
        }
        int count = 0;
        for(int i = 0, k = 0; i < rows; ++i) {
            // count how many words one row has and move to start of next row.
            // It's better to check if d[k] == k but I find there is no test case on it.
            // if(dp[k] == k) return 0;
            count += dp[k] - k;
            k = dp[k] % n;
        }
        // divide by the number of words in sentence
        return count / n;
    }
}
```

Your previous code was restored from your local storage.

Console Contribute

Problems Pick One Prev 418/1490 Next

Run Code Submit